

A Formal Security Analysis of a MACsec Key Agreement Protocol Using Tamarin

Halil İbrahim Kaplan¹[0009-0006-0120-6512]

Cryptanalysis Laboratory, TÜBİTAK BİLGEM, Kocaeli, Turkey
halil.kaplan@tubitak.gov.tr

Abstract. MACsec Key Agreement (MKA) is the IEEE 802.1X key-management protocol used to establish and maintain Secure Associations for MACsec deployments. Although MKA is widely deployed, machine-checked analyses of its core key-agreement logic remain scarce. This paper presents a formal analysis of a simplified two-party MKA exchange using the Tamarin prover. We model the initial session establishment and a subsequent rekey round, and verify secrecy, authentication, agreement, ordering, and freshness properties. The analysis confirms these guarantees under a Dolev–Yao adversary when the pre-shared Connectivity Association Key (CAK) is not compromised. We also identify a structural weakness: a malicious or compromised Key Server can inject an arbitrary Secure Association Key (SAK) that the Server accepts. This finding clarifies the trust assumptions of MKA and motivates additional verification or binding mechanisms for partially trusted deployments.

1 Introduction

The increasing deployment of Ethernet-based infrastructures has weakened the assumption that the data-link layer is inherently trustworthy. In these settings, link-layer traffic is exposed to adversarial conditions, and cryptographic protection is needed to preserve confidentiality, integrity, and replay resistance. The IEEE 802.1AE MACsec standard [1] addresses this need by defining hop-by-hop protection for Ethernet frames, while the MACsec Key Agreement protocol (MKA) [2] manages the establishment and maintenance of the cryptographic state used by MACsec.

This paper addresses that gap by presenting a formal analysis of a simplified two-party MKA exchange using the Tamarin prover. Our model captures the initial session establishment and a subsequent rekey round, and we verify essential security properties under a standard Dolev–Yao adversary model. The analysis confirms the expected guarantees when the CAK remains uncompromised, while also revealing a structural weakness in the protocol’s acceptance logic.

Contributions. This work makes three contributions. First, it constructs a Tamarin model of a two-party MKA exchange that captures the initial session and a rekey round, including the monotonic MN/KN counters. Second, it formalizes

and verifies core security properties such as executability, agreement, authentication, secrecy, ordering, and freshness. Third, it identifies and demonstrates a structural weakness in the protocol’s acceptance logic: a malicious or compromised Key Server that possesses the CAK can inject an arbitrary SAK that the Server accepts.

1.1 Related Work

Formal verification has become standard for protocol analysis. Classical examples include Lowe’s analysis of Needham–Schroeder with FDR [3], Blanchet’s ProVerif-based work on authentication protocols [4], and later Tamarin studies of TLS 1.3 [5], Signal [6], and WPA2 [7]. Within the IEEE 802 family, the closest related work concerns WPA2’s 4-way handshake; however, the MKA key-agreement logic has received comparatively little machine-checked analysis.

Our contribution is therefore complementary. To the best of our knowledge, this is the first machine-verified symbolic analysis of a simplified MKA exchange. The closest methodological precedent is the Tamarin analysis of TLS 1.3 by Cremers et al. [5], which similarly verifies authentication, secrecy, and ordering properties. Our work applies the same general methodology to the Ethernet security domain, where the role of the Key Server introduces a distinct acceptance semantics that is not present in classical protocol examples.

Organization. Section 2 provides an overview of MKA and its security requirements. Section 3 introduces the Tamarin Prover framework. Section 4 presents our formal Tamarin model and details the fourteen security lemmas. Finally, Section 5 concludes the paper by synthesizing the verification results, structural trust limitations, and future research directions.

2 MKA Overview

2.1 General Structure of MKA

IEEE 802.1AE (MACsec) [1] provides link-layer confidentiality, integrity, and replay protection for Ethernet frames on point-to-point and multi-point topologies. While MACsec specifies frame-level encapsulation and hardware/driver data-plane protection, the associated control-plane protocol specified in IEEE 802.1X-2020 [2] is the MACsec Key Agreement (MKA) protocol. MKA operates over EAP over LAN (EAPoL) control frames (EtherType 0x888E) and is responsible for establishing, discovering, authenticating, and maintaining the shared cryptographic keys—specifically the Secure Association Keys (SAKs)—required by MACsec.

The general protocol lifecycle of MKA consists of the following key phases:

1. **Connectivity Association setup.** Participants sharing identical administrative security policies form a *Connectivity Association* (CA). All members of a CA share a long-term secret key known as the *Connectivity Association Key* (CAK). The CAK is either statically pre-provisioned (PSK mode)

or dynamically derived via an Extensible Authentication Protocol (EAP) session (e.g., EAP-TLS) with an authentication server.

2. **Key Derivation Function (KDF) initialization.** Using a standardized Key Derivation Function (typically based on AES-CMAC-128 or HMAC-SHA256), each participant independently derives two operational sub-keys from the shared CAK:
 - *Key Encryption Key* (KEK): Used by the Key Server to encrypt distributed SAKs.
 - *Integrity Check Value Key* (ICK): Used to authenticate all MKA Protocol Data Units (MKAPDUs) by appending a MAC-based Integrity Check Value (ICV).
3. **Key Server election.** Rather than performing pairwise negotiations, MKA designates a single participant as the *Key Server*. Election is performed deterministically based on member priority fields and numeric Secure Channel Identifiers (SCI) acting as tie-breakers. The elected Key Server assumes exclusive responsibility for generating and distributing operational SAKs.
4. **Periodic MKAPDU exchange and peer management.** Members continuously broadcast MKAPDUs carrying Parameter Sets. Every participant maintains a *Live Peer List* and a *Potential Peer List* to monitor peer availability and verify bidirectional connectivity. Each MKAPDU includes a strictly increasing *Message Number* (MN) counter to prevent message replay.
5. **SAK distribution.** The Key Server generates a fresh symmetric key (SAK) for MACsec data encryption, encrypts it under the KEK, attaches the active *Key Number* (KN), computes an ICV using the ICK, and transmits the resulting MKAPDU to CA members. Upon receiving the frame, non-Key Server participants verify the ICV, validate the MN counter, decrypt the SAK, and confirm activation by acknowledging receipt.
6. **Periodic and event-driven rekeying.** SAKs must be refreshed periodically before sequence number exhaustion in MACsec frames or when member membership changes (e.g., peer departure). Rekeying follows the distribution structure, advancing the *Key Number* (KN) monotonically to maintain key freshness and synchronization.

The simplified Tamarin-based protocol flow shown in Figure 1 illustrates the two-party MKA exchange between a Key Server and a Server analyzed in this paper, mapping directly to the rules and state facts of our formal model.

2.2 Security Requirements of MKA

To ensure secure operating guarantees across untrusted Ethernet links, MKA must fulfill a precise set of cryptographic and protocol-level security requirements. Based on the specifications in IEEE 802.1X-2020 [2] and established formal protocol analysis standards, we define seven key security requirements:

- SR1. Protocol Executability and Reachability:** Under benign network conditions (without active adversarial blocking), the protocol execution must be reachable. Legitimate Key Server and participant nodes must successfully

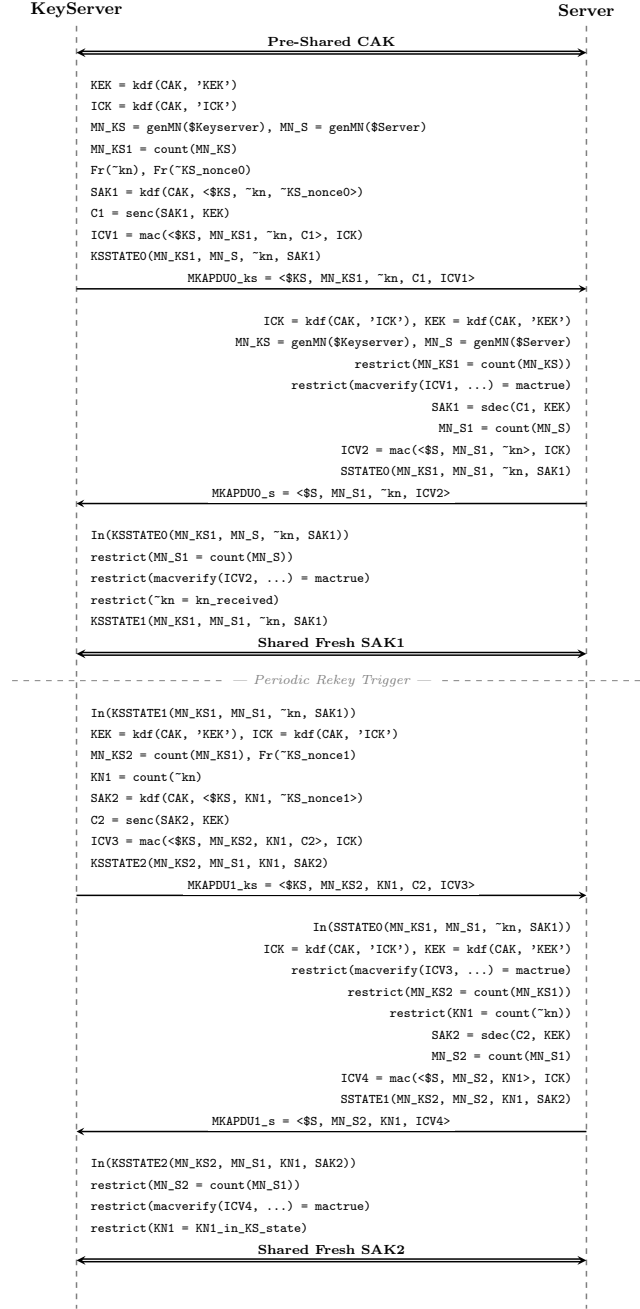


Fig. 1. Simplified two-party MKA protocol flow (initial session and rekey round) as specified in our Tamarin model. \$KS = KeyServer identity, \$S = Server identity, MN = Message Number counter, KN = Key Number counter.

complete both initial session establishment and subsequent rekeying rounds without encountering deadlocks or invalid state transitions.

- SR2. Entity Authentication and Mutual Agreement:** The protocol must guarantee mutual entity authentication and parameter agreement. When a participant completes a session believing it communicated with a specific Key Server (and vice versa), the counterpart must have actively initiated and participated in the corresponding protocol execution with identical parameters, including node identities, Key Numbers (KN), and Message Numbers (MN).
- SR3. SAK Confidentiality and Secrecy:** Operational SAKs generated by the Key Server and distributed to participants must remain confidential. An adversary controlling the public network (Dolev–Yao model) must be unable to infer or compute any SAK value, provided the long-term CAK and the target SAK have not been explicitly compromised.
- SR4. Session Ordering and Sequential Integrity:** Session state transitions must adhere strictly to protocol sequencing. Rekeying phases must be strictly conditioned on the successful completion of the preceding initial session. Out-of-order execution, premature rekeying, or backward state rollbacks must be impossible.
- SR5. SAK Freshness and Key Uniqueness:** Every rekeying round must produce a fresh, cryptographically independent SAK ($SAK_2 \neq SAK_1$). Reusing old SAKs across distinct sessions or rekey rounds must be strictly prohibited, and every newly established key must be bound to a monotonically advancing Key Number ($KN_2 > KN_1$).
- SR6. Anti-Replay Protection:** The protocol must prevent replay attacks involving stale MKAPDUs. The continuous increment of Message Numbers (MN) per participant, validated against local peer state, ensures that delayed, duplicated, or replayed control frames are immediately dropped.
- SR7. Trust Boundary and Acceptance Semantics:** Non-Key Server participants accepting a distributed SAK must be assured of its cryptographic integrity and origin. The security model must evaluate whether participants can verify that a distributed SAK was legitimately generated through the prescribed Key Derivation Function (KDF) path rather than being arbitrarily chosen or manipulated by a compromised entity.

3 Formal Analysis and Tamarin Prover

The Tamarin Prover [8,9] is a widely used tool for the automated symbolic verification of security protocols. It operates in the Dolev–Yao adversary model [10]: the adversary controls the network (can intercept, replay, modify, and inject messages) but cannot break cryptographic primitives (for example, cannot invert a hash, decrypt without the key, or forge a MAC).

Modelling language. Tamarin protocols are specified as *multiset rewriting rules* over a set of *facts*. Each rule has the form

$$[\text{premise facts}] \xrightarrow{[\text{action facts}]} [\text{conclusion facts}]$$

The premise specifies what facts must be present in the current system state, action facts annotate the rule firing with observable events, and the conclusion specifies what facts are produced. The special facts $\text{Fr}(\mathbf{x})$ generate fresh nonces, $\text{In}(\mathbf{m})$ receives a network message, and $\text{Out}(\mathbf{m})$ sends one. Persistent facts (prefixed with $!$) survive consumption.

Equational theories. Tamarin supports equational theories that model algebraic properties of cryptographic operations. For instance, $\text{sdec}(\text{senc}(\mathbf{m}, \mathbf{k}), \mathbf{k}) = \mathbf{m}$ captures symmetric encryption/decryption. The tool reasons about an arbitrary Dolev-Yao adversary that can apply these equations to derive new messages.

Restrictions and lemmas. Restrictions (formerly called axioms) constrain the set of valid execution traces, allowing the modeller to express protocol-specific validity conditions (e.g., “a MAC must verify”). Lemmas specify security properties in a fragment of first-order logic with time annotations:

- **all-traces** lemmas express universal properties (safety, authentication, secrecy).
- **exists-trace** lemmas assert executability or demonstrate attack traces.

Tamarin employs a constraint-solving procedure to discharge lemmas. It can run with manual or automatic heuristics. Its soundness and completeness guarantees hold relative to the modelled equational theory.

Debugging and reproducibility. When a lemma fails, Tamarin produces a concrete counterexample trace. This trace can be exported to LaTeX via the `-trace` option, which generates a `trace.tex` file that can be directly included in the paper. For reproducibility, all proof outputs—including `summary.pdf`, `trace.tex`, and the exact command-line invocation—are archived alongside the model in the repository.

4 Tamarin Model

4.1 Cryptographic Primitives and Equational Theory

Our Tamarin model is built on the following builtins and functions:

- **Symmetric encryption/decryption:** We used predefined functions `senc/sdec` satisfying $\text{sdec}(\text{senc}(\mathbf{m}, \mathbf{k}), \mathbf{k}) = \mathbf{m}$ for symmetric encryption and decryption. The adversary can only decrypt a ciphertext if it knows the corresponding key.
- **Key derivation:** `kdf(key, label)` is modelled as an uninterpreted two-argument function; no algebraic identities are assumed, which captures the idealised one-way property of a KDF.

- **Message authentication:** Tamarin does not provide a built-in MAC primitive, so we define $\text{mac}(M, K)$ and $\text{macverify}(\text{mac}(M, K), M, K) = \text{mactrue}$; the equation ensures that the adversary can only produce a valid ICV if it knows the ICK.
- **Monotone counters:** We define $\text{count}(x)$ to model counter advancement and use it for both MN and KN progression.
- **Message-number generation:** We define a private function $\text{genMN}(\$Party)$ that produces a deterministic initial message number for each participant identity. The function is declared *private*, meaning the adversary cannot compute or predict initial MN values. This models the assumption that both participants have already exchanged at least one MKAPDU in a prior discovery phase and maintain private MN state. This abstraction allows our model to focus on the key agreement phase without modeling the full MKA hello/discovery protocol.

Inline restrictions. Tamarin supports two mechanisms for constraining execution traces: named **restriction** blocks and inline **_restrict** action facts. Our model uses inline **_restrict** actions within protocol rules to enforce local validity conditions (e.g., MAC verification, counter monotonicity) directly at the point where they are checked. This approach keeps each rule self-contained and mirrors the sequential checking logic of the actual protocol implementation.

4.2 Protocol Participants and Long-Term State

CAK setup. The rule **Setup_CAK** generates a fresh CAK cak and stores it as a persistent shared fact $!SharedCAK(\sim cak)$, available to both parties.

Derived keys. Both participants independently compute

$$kek = \text{kdf}(cak, 'KEK'), \quad ick = \text{kdf}(cak, 'ICK')$$

at the beginning of each protocol step, since both parties share the same CAK.

4.3 Initial Session Rules

Initial_KeyServer_Send. The Key Server:

1. Retrieves $!SharedCAK(\sim cak)$.
2. Generates a fresh Key Number kn and a fresh nonce KS_nonce0 .
3. Derives $sak1 = \text{kdf}(cak, \langle \$Keyserver, kn, KS_nonce0 \rangle)$.
4. Computes $c1 = \text{senc}(sak1, kek)$.
5. Computes $icv1 = \text{mac}(\langle \$Keyserver, mn_ks1, kn, c1 \rangle, ick)$.
6. Sends $\langle \$Keyserver, mn_ks1, kn, c1, icv1 \rangle$ and stores $KSSTATE0$.

The action fact **SentInitialSAK** is recorded for use in the authentication lemmas.

Initial_Server_Receive. The Server:

1. Receives the MKAPDU from the network.
2. Retrieves $\text{!SharedCAK}(\sim\text{cak})$ and verifies icv1 using a `_restrict` action:
 $\text{macverify}(\text{icv1}, \dots, \text{ick}) = \text{mactrue}.$
3. Restricts the received MN to match the expected counter: $\text{mn_ks1} = \text{count}(\text{mn_ks}).$
4. Decrypts the payload: $\text{sak1} = \text{sdec}(c1, \text{kek}).$
5. Computes and sends its acknowledgement ICV2.
6. Records `ServerAcceptedSAK` and stores `SSTATE0`.

Initial_KeyServer_Confirm. The Key Server receives the Server's ACK, verifies ICV2, checks that the received MN matches the expected counter, and transitions to `KSSTATE1`, recording `InitialSessionCompleted`.

4.4 Rekey Rules

The rekey phase mirrors the initial phase structurally, but it reads the previous state (`KSSTATE1/SSTATE0`) to enforce monotonicity. Specifically:

- $\text{mn_ks2} = \text{count}(\text{mn_ks1})$ and $\text{kn1} = \text{count}(\text{kn})$ ensure both counters advance.
- A fresh nonce KS_nonce1 ensures SAK2 is independent of SAK1 even for identical KN paths.

4.5 Adversary Model and Compromise Rules

We model a standard Dolev–Yao adversary augmented with two compromise rules:

- `Reveal_SAK`: exposes an initial SAK from `KSSTATE0`; records `RevealSAK`.
- `Reveal_CAK`: exposes the CAK from `!SharedCAK`; records `RevealCAK`.

These rules allow the formulation of conditional security properties, for example that secrecy holds unless the CAK is revealed.

The rule `Malicious_KeyServer_Send` models a Key Server that uses the CAK to construct a well-formed MKAPDU carrying an arbitrary fresh SAK (fakeSAK) instead of a properly KDF-derived one. This rule is used to probe the protocol's acceptance mechanism.

Forward-secrecy limitation. Because all operational keys (KEK, ICK, SAK) are derived from the single long-term CAK, compromising the CAK at any point enables an adversary to re-derive all past and future sub-keys. The protocol therefore does not provide forward secrecy: a CAK compromise retroactively exposes every session. This is an inherent limitation of pure pre-shared-key architectures without ephemeral key exchange, and it is captured by the conditional form of our secrecy lemmas, which explicitly exclude CAK-reveal traces.

Table 1. Security lemmas verified against the MKA model. The SR column maps each lemma to the security requirements defined in Section 2.2.

Lemma	Type	Property	SR	Res.
executable	exists-trace	Executability	SR1	✓
initial_agreement	all-traces	Initial agree- ment	SR2	✓
rekey_agreement	all-traces	Rekey agree- ment	SR2	✓
server_authentication_initial	all-traces	Server aliveness	SR2	✓
server_authentication_rekey	all-traces	Rekey aliveness	SR2	✓
initial_sak_secretcy_no_reveal	all-traces	SAK1 secrecy	SR3	✓
rekey_sak_secretcy_no_reveal	all-traces	SAK2 secrecy	SR3	✓
rekey_cannot_start_before_	all-traces	Ordering	SR4	✓
initial_completion				
rekey_after_initial_completion	all-traces	Ordering	SR4	✓
key_update	all-traces	SAK1 $\neq$ SAK2	SR5	✓
key_number_progress	all-traces	KN monotonic- ity	SR5,6	✓
fresh_rekey_key	all-traces	Rekey freshness	SR5	✓
server_only_accepts_kdf_keys	exists-trace	KDF bypass	SR7	✓
accepts_non_kdf_sak	exists-trace	Malicious SAK	SR7	✓

Regarding quantum adversaries. Our symbolic model abstracts away computational complexity and therefore does not capture brute-force attacks or their quantum speedups (e.g., Grover’s algorithm reducing effective symmetric key strength from 2^{128} to 2^{64} operations). Since all cryptographic primitives in MKA are symmetric, the protocol is not vulnerable to Shor’s algorithm. Assessing quantum resilience of the symmetric primitives requires a computational (rather than symbolic) analysis and is outside the scope of this work.

4.6 Security Properties and Lemmas

We specify fourteen security properties as formal Tamarin lemmas. Table 1 summarizes these lemmas alongside their formal verification results and the security requirements (SR1–SR7, defined in Section 2.2) that each lemma addresses. The checked entries confirm that the protocol is executable and that all key agreement, authentication, secrecy, ordering, and freshness properties hold under the stated assumptions. Crucially, the two existence lemmas confirm that the model permits a valid protocol execution while simultaneously uncovering a structural acceptance weakness: a malicious or compromised Key Server holding the CAK can distribute an arbitrary, non-KDF-derived SAK that the receiver accepts.

The fourteen Tamarin lemmas defined in our model fall into six functional categories, formalizing the security requirements defined in Section 2.2. In Tamarin syntax, temporal markers $\#i$ and $\#j$ denote time points on an execution trace (where $@i$ indicates an event occurring at time $\#i$), and $K(m)$ denotes the adver-

sary's knowledge of message m . Below, we present the formal specifications and semantic analysis of each lemma group.

Protocol Executability To guarantee that the protocol rules do not contain logical deadlocks or unsatisfiable restrictions, we verify the reachability of a full two-round execution trace.

*Executable Lemma (**executable**)*. Formulated as an **exists-trace** property, this lemma asserts that there exists at least one execution trace where both the initial session and a subsequent rekey round complete successfully:

$$\begin{aligned} & \exists mn_{ks1}, mn_{s1}, kn, sak1, mn_{ks2}, mn_{s2}, kn1, sak2, i, j. \\ & \text{InitialSessionCompleted}(mn_{ks1}, mn_{s1}, kn, sak1)@i \wedge \\ & \text{RekeySessionCompleted}(mn_{ks2}, mn_{s2}, kn1, sak2)@j \end{aligned}$$

Tamarin successfully synthesizes a valid trace, proving that the model state facts and counter restrictions permit complete protocol runs under non-adversarial conditions (fulfilling **SR1**).

Agreement and Entity Authentication Mutual agreement lemmas ensure that both parties share consistent view of session parameters and that no session completes without the active participation of the counterparty.

*Initial and Rekey Agreement (**initial_agreement**, **rekey_agreement**)*. These **all-traces** lemmas enforce injective agreement from the Key Server's perspective. For any trace where an initial (or rekey) session completes at time i , the Key Server must have previously initiated and transmitted that exact SAK at an earlier time $j < i$:

$$\begin{aligned} \forall mn_{ks1}, mn_{s1}, kn, sak1, i. \text{InitialSessionCompleted}(mn_{ks1}, mn_{s1}, kn, sak1)@i \implies \\ (\exists j. \text{SentInitialSAK}(mn_{ks1}, kn, sak1)@j \wedge j < i) \end{aligned}$$

Server Authentication Lemmas. The lemmas **server_authentication_initial** and **server_authentication_rekey** establish non-injective aliveness of the Server node. Provided that the shared long-term CAK has not been compromised ($\neg \exists cak, k. \text{RevealCAK}(cak)@k$), the Key Server's completion of an initial or rekey session guarantees that the Server actively validated the MKAPDU and prepared a corresponding acknowledgement ACK at time $j < i$ (fulfilling **SR2**).

SAK Confidentiality and Secrecy Secrecy lemmas verify that operational SAKs derived during protocol runs remain protected against passive and active Dolev–Yao adversaries.

SAK Secrecy Lemmas. The secrecy lemmas (`initial_sak_secrecy_no_reveal` and `rekey_sak_secrecy_no_reveal`) state that if a session completes at time i , and neither the session key ($sak1$ or $sak2$) nor the long-term secret (cak) has been exposed via adversary reveal rules (`RevealSAK` or `RevealCAK`), then the adversary can never learn or derive the SAK:

$$\begin{aligned} \forall mn_{ks1}, mn_{s1}, kn, sak1, i. [\text{InitialSessionCompleted}(mn_{ks1}, mn_{s1}, kn, sak1)@i \wedge \\ \neg(\exists k. \text{RevealSAK}(mn_{ks1}, mn_{s1}, kn, sak1)@k) \wedge \neg(\exists cak, l. \text{RevealCAK}(cak)@l)] \implies \\ \neg(\exists m. \mathbf{K}(sak1)@m) \end{aligned}$$

Both secrecy lemmas hold, proving that SAK confidentiality is maintained under uncompromised CAK conditions (fulfilling **SR3**).

Session Ordering and Transition Integrity Ordering lemmas enforce state-machine flow rules, preventing out-of-order execution or state rollback.

Pre-Rekey Ordering. The lemma `rekey_cannot_start_before_initial_completion` asserts that a Key Server cannot initiate a rekey round (`RekeyStarted`) at time i unless an initial session has already been completed at time $j < i$.

Rekey Completion Ordering (`rekey_after_initial_completion`). Asserts that a rekey session cannot complete at time i without a prior initial session completion at time $j < i$. Together, these two `all-traces` lemmas confirm that protocol execution is strictly sequential (fulfilling **SR4**).

Key Monotonicity and Rekey Freshness Freshness lemmas ensure that key updates produce distinct, un-reused cryptographic keys bound to advancing sequence numbers.

Key Update Lemma (`key_update`). Proves that for any execution trace containing an initial session completion at time i and a rekey completion at time j , the rekeyed key $sak2$ is strictly non-identical to the initial key $sak1$ ($\neg(sak1 = sak2)$).

Key Number Progress (`key_number_progress`). Confirms counter monotonicity by proving that the Key Number of a rekeyed session ($kn1$) is equal to the successor count of the initial Key Number ($kn1 = \text{count}(kn)$).

Fresh Rekey Key (`fresh_rekey_key`). Ensures key freshness by verifying that a rekeyed SAK ($sak2$) accepted in a rekey session could never have been distributed as a valid key in any initial session. These lemmas collectively satisfy **SR5** and **SR6**.

KDF-Bypass and Malicious Key Server Acceptance To probe the trust boundary of the protocol (**SR7**), we evaluate two `exists-trace` lemmas using the `Malicious_KeyServer_Send` rule.

KDF Bypass Lemma (*server_only_accepts_kdf_keys*). Evaluates whether there exists a trace where the Server accepts a key ($\text{ServerAcceptedSAK}(sak)$) that was never generated via the standard KDF path ($\text{KDFGeneratedSAK}(sak)$).

Malicious SAK Acceptance Lemma (*accepts_non_kdf_sak*). Synthesizes an explicit attack trace where a malicious Key Server generates an arbitrary key $\widetilde{fakeSAK}$ and embeds it in a well-formed MKAPDU:

$$\exists sak, i, j. \text{MaliciousSAKSent}(sak)@i \wedge \text{ServerAcceptedSAK}(sak)@j \wedge i < j$$

Tamarin finds a valid trace for both lemmas, confirming that non-KDF key acceptance is achievable in the model. Listing 1.1 shows the Tamarin source for these two critical components.

```
rule Malicious_KeyServer_Send:
  let kek = kdf(~cak, 'KEK')
  ick = kdf(~cak, 'ICK')
  mn_ks = genMN($Keyserver)
  mn_ks1 = count(mn_ks)
  c1 = senc(~fakeSAK, kek)
  icv1 = mac(<$Keyserver, mn_ks1, ~kn, c1>, ick)
in
[ !SharedCAK(~cak), Fr(~fakeSAK), Fr(~kn) ]
--[ MaliciousSAKSent(~fakeSAK) ]->
[ Out(<$Keyserver, mn_ks1, ~kn, c1, icv1>) ]

lemma accepts_non_kdf_sak: exists-trace
"Ex sak #i #j.
  MaliciousSAKSent(sak)@i
  & ServerAcceptedSAK(sak)@j
  & #i < #j"
```

Listing 1.1. Malicious Key Server rule and KDF-bypass existence lemma in Tamarin syntax.

4.7 Structural Weakness: KDF-Bypass Attack

Definition 1 (Malicious SAK Acceptance). *There exists a trace in which $\text{MaliciousSAKSent}(sak)$ is observed before $\text{ServerAcceptedSAK}(sak)$, and sak was never output by KDFGeneratedSAK .*

This exists-trace lemma captures the scenario in which a Key Server holding the CAK injects a fresh but non-KDF-derived SAK into a well-formed MKAPDU. The concrete attack trace proceeds as follows:

1. The malicious Key Server, possessing the shared CAK, derives the operational sub-keys $kek = \text{kdf}(cak, 'KEK')$ and $ick = \text{kdf}(cak, 'ICK')$.
2. Instead of computing a SAK via the prescribed KDF path ($\text{kdf}(cak, \langle \$KS, kn, nonce \rangle)$), it generates an arbitrary fresh value $\widetilde{fakeSAK}$.

3. It encrypts the arbitrary key under the shared KEK: $c = \text{senc}(\widetilde{\text{fakeSAK}}, \text{kek})$.
4. It computes a valid ICV over the MKAPDU fields using the shared ICK: $\text{icv} = \text{mac}(\langle \$KS, mn, kn, c \rangle, \text{ick})$.
5. The Server receives the MKAPDU, verifies the ICV (which succeeds because the ICK is correctly derived from the shared CAK), validates the MN counter, and decrypts $\widetilde{\text{fakeSAK}}$ from the ciphertext.
6. The Server accepts $\widetilde{\text{fakeSAK}}$ as the session key, with no mechanism to detect that it was not derived via the prescribed KDF path.

Practical impact. A Key Server that chooses the SAK gains complete control over the data-plane encryption key. Since MACsec uses the SAK for symmetric authenticated encryption of Ethernet frames, an adversary who selected the SAK value can:

- Decrypt all traffic protected under that SAK (passive eavesdropping).
- Inject or modify MACsec-protected frames (active attacks).
- Share the chosen SAK with a third party, enabling a relay or wiretap without the Server’s knowledge.

Root cause and mitigations. The weakness arises because the MKAPDU authentication (via the ICV) guarantees that the sender possesses the CAK, but does not bind the SAK payload to the prescribed derivation path. Potential mitigations include:

- **KDF proof-of-compliance:** The Key Server could include the KDF inputs (Key Number, nonce) in a verifiable form, allowing the Server to independently re-derive and compare the SAK. However, this requires the Server to have access to the KDF nonce, which is not currently transmitted in the clear.
- **Mutual key derivation:** Both parties could contribute entropy to the SAK derivation (e.g., via a Diffie–Hellman-like exchange or by each providing a nonce), ensuring no single party controls the key value. This approach would require protocol-level changes to IEEE 802.1X.
- **Key confirmation:** A post-distribution key-confirmation step in which both parties demonstrate knowledge of the SAK through a challenge–response mechanism could detect mismatches, although it would not prevent a malicious Key Server from knowing the key it injected.

This is a *structural* property of the protocol rather than a cryptographic break: all cryptographic primitives function correctly, but the protocol’s trust model implicitly assumes that the Key Server is honest.

4.8 Gap between IEEE 802.1X-2020 and our model

While our Tamarin model successfully verifies MKA under a standard Dolev-Yao adversary, the model necessarily abstracts away several operational details present in the IEEE 802.1X-2020 standard. The primary gaps include:

- **Keyserver Selection:** We omit the keyserver selection process and assume the keyserver is already selected. This assumption is valid for our model because we are primarily concerned with the security of the key agreement protocol itself, not the keyserver discovery process.
- **Detailed State Machines:** IEEE 802.1X-2020 specifies comprehensive finite-state machines for key generation, SA activation, and session state transitions. Our model captures the high-level logic (initiation, key exchange, rekeying) but omits low-level transitions like timers, retry counts, and error states.
- **Message Numbering:** The IEEE 802.1X-2020 standard specifies a message numbering scheme for MKAPDU messages. We assumed that both participants already send MKAPDUs; therefore, they know all participants’ message numbers and can compare new message numbers with previous ones. This assumption is valid in our model because we assume that both participants have already exchanged MKAPDUs and know each other’s message numbers.

4.9 Proof Statistics and Reproducibility

All fourteen lemmas were verified using Tamarin Prover version `TODO_VERSION` on a machine equipped with a `TODO_CPU` processor and `TODO_RAM` GB of RAM, running `TODO_OS`. Proofs were discharged using Tamarin’s automatic mode with the default heuristic (`TODO_HEURISTIC`). The total verification time for all fourteen lemmas was approximately `TODO_TIME`. No manual proof guidance (i.e., oracle or interactive mode) was required; all lemmas terminated automatically.

The complete Tamarin model, including all lemma definitions and auxiliary scripts, is publicly available at https://github.com/KaplanHalil/MKA_tamarin for full reproducibility.

4.10 Verification Summary

The verification confirms that MKA provides the expected security guarantees when the long-term CAK is uncompromised. Specifically: (i) the model is executable and free of logical deadlocks (SR1); (ii) the Key Server and Server achieve mutual agreement on session parameters (SR2); (iii) SAKs remain confidential under the Dolev–Yao adversary (SR3); (iv) session ordering is strictly sequential (SR4); and (v) rekeyed keys are fresh, distinct, and bound to monotonically advancing counters (SR5, SR6).

The most significant finding is the existence of an attack trace confirming the KDF-bypass weakness described in Section 4.7 (SR7). A malicious Key Server holding the CAK can construct a well-formed MKAPDU carrying an arbitrary fresh SAK. Because the MAC is computed with the ICK derived from the shared CAK, the Server accepts the message and records the injected SAK as valid. This result exposes a structural limitation in the protocol’s trust model: the Server has no independent proof that the SAK was generated through the prescribed KDF path.

5 Conclusion

In this paper, we presented a formal symbolic security analysis of the MACsec Key Agreement (MKA) protocol using the Tamarin Prover. Our model captures a two-party MKA exchange, incorporating natural-number counter abstractions for Message Numbers (MN) and Key Numbers (KN), and covers both the initial SAK distribution and a subsequent rekey round.

We formally specified and verified fourteen security properties under a standard Dolev–Yao adversary model. The analysis proves that when the long-term Connectivity Association Key (CAK) is uncompromised, MKA successfully guarantees protocol executability, mutual agreement on session parameters, SAK confidentiality, strict session ordering, and key freshness across rekeying rounds.

At the same time, our verification reveals a structural trust limitation in MKA’s acceptance logic: a compromised or malicious Key Server possessing the CAK can inject an arbitrary SAK value that the Server accepts. Because non-Key Server participants rely solely on ICV verification using the derived ICK, they cannot independently verify whether an accepted SAK originated from the standard KDF path. This underlines an implicit trust assumption in IEEE 802.1X-2020 and motivates future work on cryptographic binding mechanisms for SAK derivation.

Future Work. Future directions include extending the model to multi-party MKA topologies, modeling dynamic peer lists and election logic, and combining symbolic verification with computational proofs. Our complete Tamarin model is publicly available at https://github.com/KaplanHalil/MKA_tamarin.

References

1. “IEEE standard for local and metropolitan area networks—MAC security (MACsec),” IEEE Std 802.1AE-2018, IEEE, 2018.
2. “IEEE standard for local and metropolitan area network—port-based network access control,” IEEE Std 802.1X-2020, IEEE, 2020.
3. G. Lowe, “Breaking and fixing the Needham-Schroeder public-key protocol using FDR,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 1996)*. Springer, 1996, pp. 147–166.
4. B. Blanchet, “An efficient cryptographic protocol verifier based on Prolog rules,” in *14th IEEE Computer Security Foundations Workshop (CSFW)*. IEEE, 2001, pp. 82–96.
5. C. Cremers, M. Horvat, S. Scott, and T. van der Merwe, “A comprehensive symbolic analysis of TLS 1.3,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1773–1788.
6. N. Kobeissi, K. Bhargavan, and B. Blanchet, “Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach,” in *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2017, pp. 435–450.
7. M. Vanhoef and F. Piessens, “Key reinstallation attacks: Forcing nonce reuse in WPA2,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1313–1328.

8. S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN prover for the symbolic analysis of security protocols,” in *Computer Aided Verification (CAV 2013)*. Springer, 2013, pp. 696–701.
9. D. Basin, J. Dreier, and R. Sasse, “Automated verification of security protocols with global state,” in *Proceedings of the 22nd ACM Conference on Computer and Communications Security (CCS)*. ACM, 2015, p. I.
10. D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.